



Ing. Fernando Negozi

fernando.negozi@eletica.it

www.eletica.it



Introduzione

File contenente un elenco di comandi di sistema da eseguire in sequenza

-linguaggio di programmazione -

Vantaggi dello script :

Modificato

Personalizzato

Riutilizzabile

Possiamo: gestire file,
 monitorare/gestire il disco ed i processi
 Gestire e controllare l'accesso alla rete

Attenzione: si può corrompere il proprio ambiente (se si usano diritti di superutente)



SCRIPT BASH

Script Language

La shell interpreta comandi interni e può richiedere l'esecuzione di comandi esterni.

comandi interni + operatori aritmetico logici + metacaratteri + file eseguibili = Scripting language

Comandi interni
Interpretati dalla shell stessa

Comandi esterni
Viene ricercato il file eseguibile.
Caricato in memoria.
Eseguito.



SCRIPT BASH - Hello world

```
#!/bin/bash
```

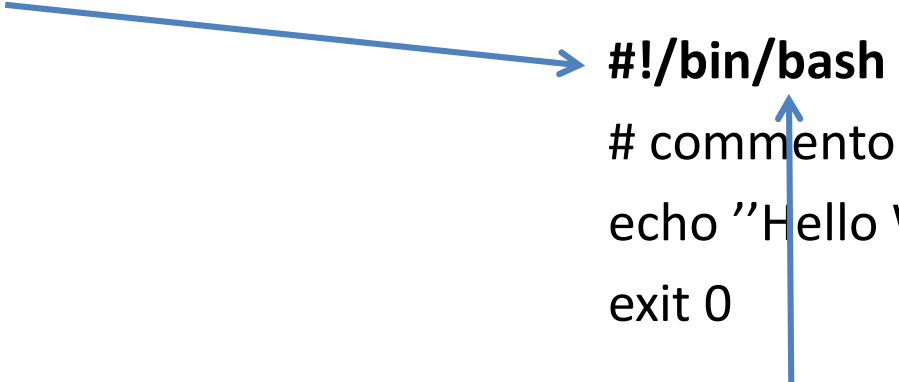
```
# commento
```

```
echo "Hello World"      # commento!
```

```
exit 0
```

Hello world

#! - Sempre sulla prima riga dello script
Informano il sistema che il file contiene una serie di comandi che devono essere passati all'interprete indicato.



```
#!/bin/bash  
# commento  
echo "Hello World"      # commento!  
exit 0
```

Percorso al programma che deve interpretare i comandi.

Informa il sistema che il file deve essere eseguito da **/bin/bash**, che corrisponde al percorso standard della Bourne-Again Shell su qualsiasi sistema Unix.

Hello world

Le righe che iniziano (eccetto la prima) con questo simbolo sono un commento non vengono eseguite

#!/bin/bash

commento

echo "Hello World"

exit 0

commento!

_ spazio

Anche al termine del comando possiamo inserire un commento con il simbolo #

Attenzione:

In altri contesti il carattere # assume altro significato

Es: "testo # testo"

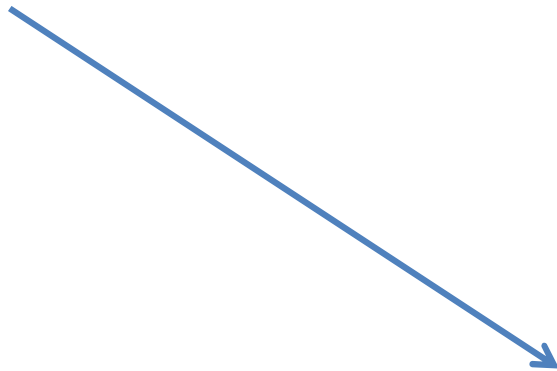
'testo # testo '

/#



Hello world

echo comando da eseguire



```
#!/bin/bash
```

```
# commento
```

```
echo "Hello World"
```

```
# commento!
```

```
exit 0
```

echo

Stampa una linea di testo nello stdout

Hello world

Exit usato per terminare lo script

Può restituire un valore
Facoltativo

Quando uno script termina con **exit** senza alcun parametro o senza exit, l'exit status dello script è quello dell'ultimo comando eseguito

```
#!/bin/bash
```

```
# commento
```

```
echo "Hello World"
```

```
# commento!
```

```
exit 0
```

Valore restituito al processo genitore

Convenzione:

0 – comando eseguito correttamente

1-255 codice (errore)

\$? legge l'exit status dell'ultimo comando eseguito.
Utilizzabile dopo lo script

Esecuzione dello script

Metodi di esecuzione di uno script

./ \$ *./nome.sh* (*cerca nella directory corrente*)

.sh \$ sh *nome.sh*

.bash \$ bash *nome.sh*

Per ragioni di sicurezza la dir corrente non è incluse in quelle di ricerca. Va indicata

NB: Deve avere i permessi di esecuzione e lettura (*)
(*) la shell lo deve leggere.

È convenzione che gli script abbiano estensione .sh



EDITOR TESTO

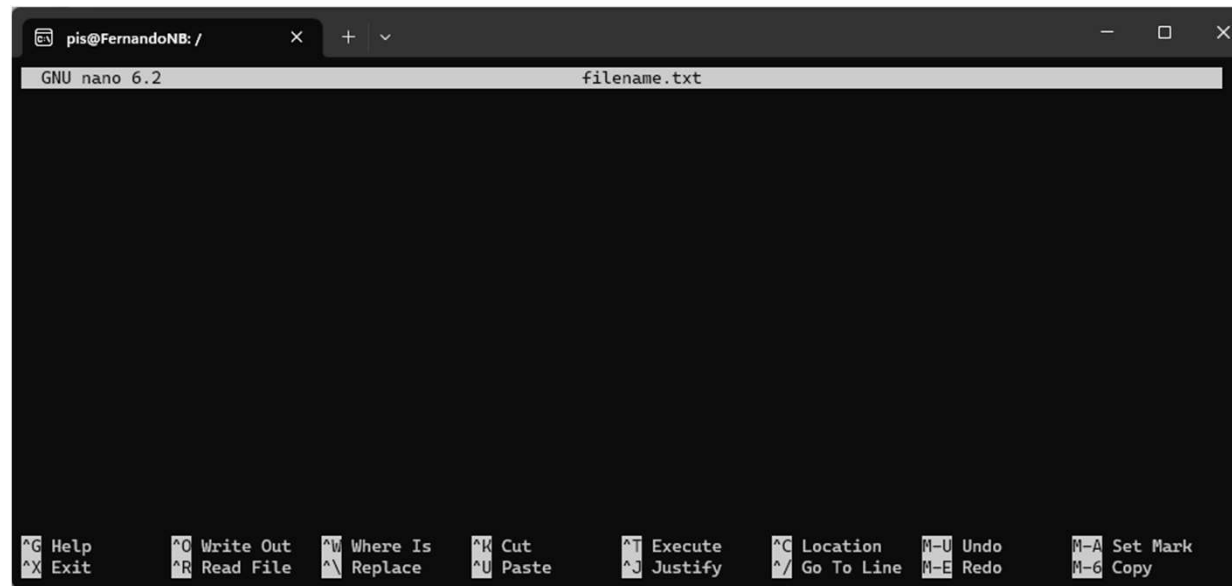
nano

editor file di testo

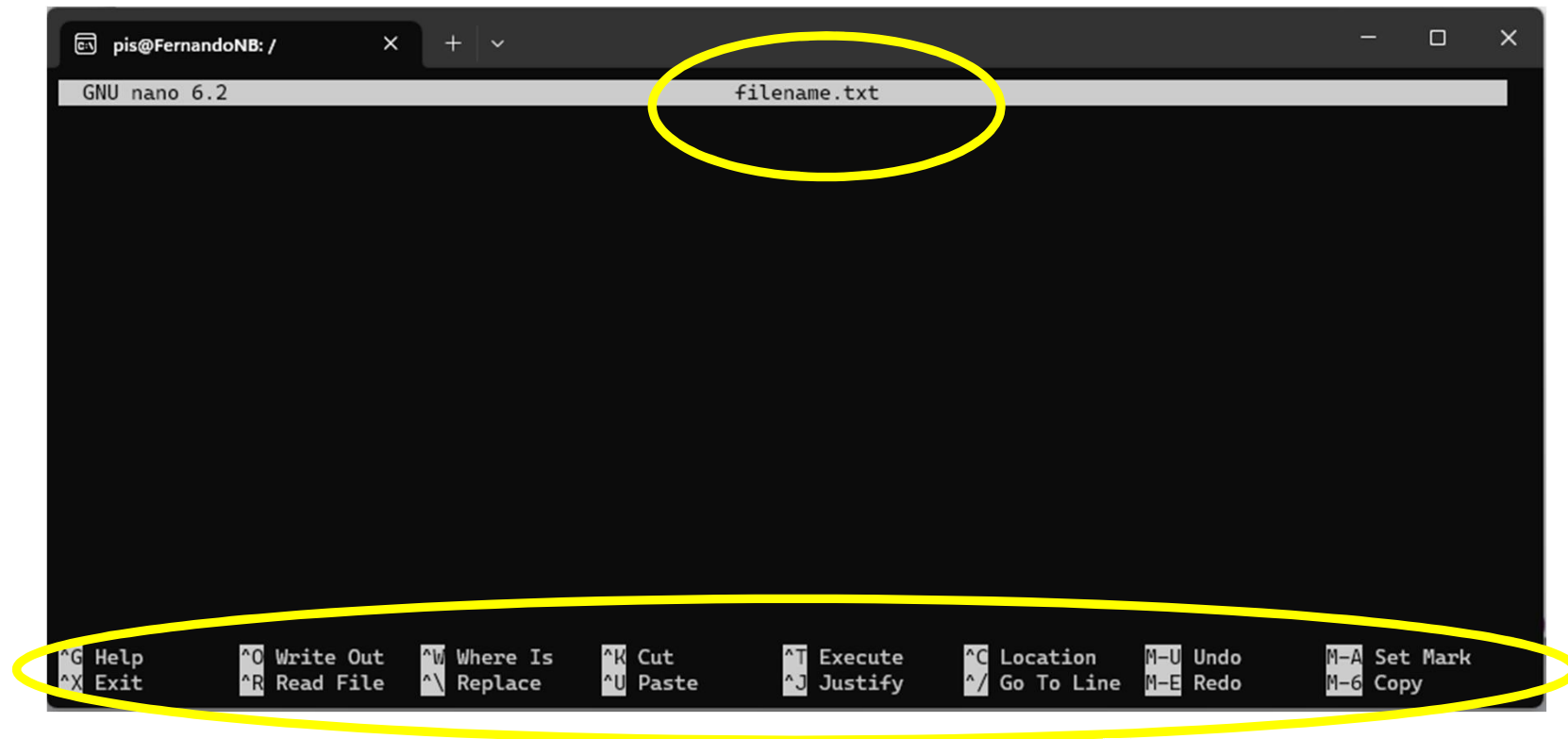
Semplice

intuitivo

sintassi: *nano /path/filename*

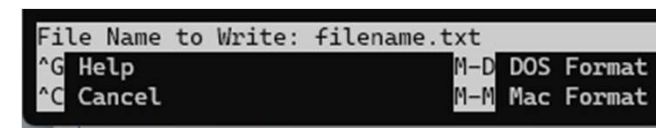
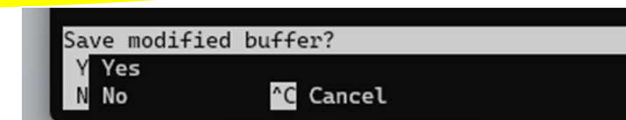


nano



Ctrl+G: apre la guida

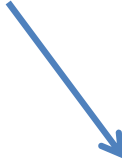
Ctrl+X: Chiude. Se ci sono modifiche chiede il salvataggio ed eventuale nome del file



Esecuzione script hello world

- Andare nella cartella utente
- Creare la sottocartella 'esercizi'
- Entrare nella cartella 'esercizi'
- Creare il file hello.sh che stampi la scritta "hello world"
- **Eseguire lo script "hello.sh"** (Attenzione!!!!)

hello.sh



```
#!/bin/bash
```

```
echo Hello World
```

```
exit
```



ESERCIZIO-SOLUZIONE

```
pis@FernandoNB: ~/esercizi  X + v
pis@FernandoNB:~$ cd
pis@FernandoNB:~$ mkdir esercizi
pis@FernandoNB:~$ cd esercizi/
pis@FernandoNB:~/esercizi$ nano hello.sh

GNU nano 6.2 hello.sh *
#!/bin/bash
echo Hello World
exit

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line

pis@FernandoNB:~/esercizi$ chmod a+x hello.sh
pis@FernandoNB:~/esercizi$ ./hello.sh
Hello World
pis@FernandoNB:~/esercizi$
```



VARIABILI

Sintassi: *nome_variabile=valore_assegnato*

Memorizza un valore o una stringa utilizzabile nel codice

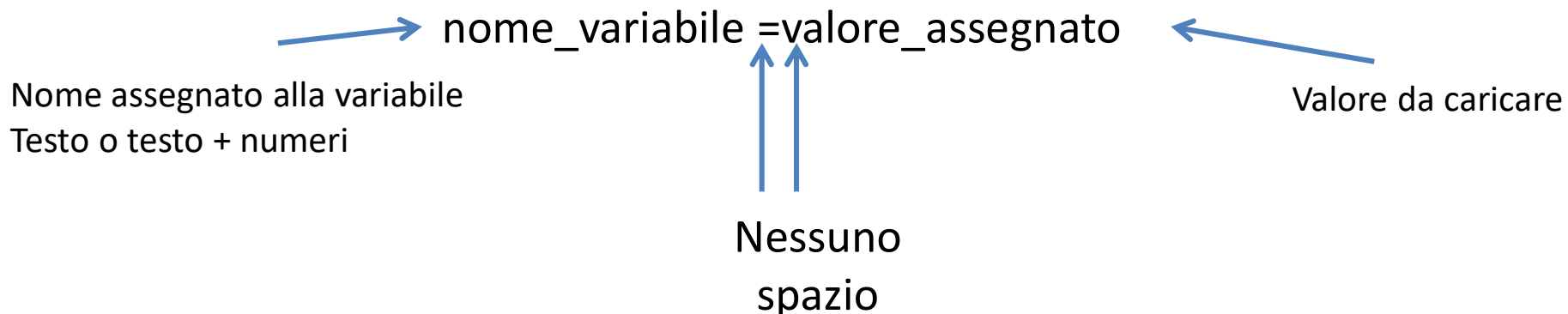
Globale all'interno della shell

A tipizzazione dinamica.

interi , stringhe , numeri reali,-.

Variabile = etichetta associata ad una porzione di memoria

- Assegnazione valore
- Prelevare valore
- Modifica valore



Esempio di utilizzo: ***echo \$nome_variabile***

Variabili BASH predefinite :

\$0 nome dello script bash

\$1 primo argomento passato allo script *./nomescript.sh arg1 arg 2 ... argN*

\$2 secondo argomento passato allo script

..... (NB: per posizioni >9 \${10})

E' possibile assegnare ad una variabile un valore di default nel caso non sia stata precedentemente inizializzata

Esempio di utilizzo: ***echo \${nome_variabile:-HELLO WORLD}***
:- Operatore di default Valore utilizzato

Esempio di utilizzo: ***echo \${nome_variabile:=HELLO WORLD}***
:= Operatore di default Valore assegnato

`./nomescript.sh arg1 arg2 arg3`

SHIFT

Sposta gli argomenti della riga di comando verso sinistra.

Dopo l'istruzione shift l'argomento che era raggiungibile con \$2 diventa raggiungibile con \$1

==> Dopo shift: \$1= arg 2

Utilizzato per scorrere tutti gli argomenti passati attraverso la riga di comando

\$@

rappresenta tutti gli argomenti della riga di comando passati a uno script Bash come array

“\$@” = (arg1 arg2 arg 3)

\$IFS

Internal Field Separator

Valore di default: SPACE NEWLINE TAB

Assegnando un valore diverso ad IFS si possono utilizzare separatori stringa differenti

Es:

Se

IFS=-

La stringa : uno-due-tre viene considerata come tre valori differenti

Sintassi: `read [options]`

Legge una linea dallo standard input

Options:

```
-a array  assign the words read to sequential indices of the array
           variable ARRAY, starting at zero
-d delim  continue until the first character of DELIM is read, rather
           than newline
-e        use Readline to obtain the line
-i text   use TEXT as the initial text for Readline
-n nchars return after reading NCHARS characters rather than waiting
           for a newline, but honor a delimiter if fewer than
           NCHARS characters are read before the delimiter
-N nchars return only after reading exactly NCHARS characters, unless
           EOF is encountered or read times out, ignoring any
           delimiter
-p prompt output the string PROMPT without a trailing newline before
           attempting to read
-r        do not allow backslashes to escape any characters
-s        do not echo input coming from a terminal
-t timeout      time out and return failure if a complete line of
           input is not read within TIMEOUT seconds. The value of the
           TMOUT variable is the default timeout. TIMEOUT may be a
           fractional number. If TIMEOUT is 0, read returns
           immediately, without trying to read any data, returning
           success only if input is available on the specified
           file descriptor. The exit status is greater than 128
           if the timeout is exceeded
-u fd       read from file descriptor FD instead of the standard input
```



Caricamento dinamico.

Sintassi: ***read TESTO_TO_READ***

Preleva dallo standard input il valore e lo mette nella variabile TEST_TO_READ

Utilizzo di un comando

Sintassi: ***NOME_VARIABLE=COMANDO***

Memorizza il comando con il nome della variabile.

Esempio:

```
variabilecomando=echo  
${variabilecomando} "Hello World"
```

Variabili di ambiente

Esiste un insieme di variabili di ambiente predefinite che contengono informazioni utili sull'ambiente di esecuzione.

Il loro valore è in genere assegnato in un file di start-up (altrimenti, la shell può usare un valore di default).

Le variabili di ambiente più usate sono:

- o \$HOME pathname della home directory
- o \$PATH lista di directory in cui ricercare un comando
- o \$USER user-id dell'utente
- o \$SHELL pathname della shell di login
- o \$TERM tipo del terminale corrente
- o \$NAME nome dell'utente

PATH

Contiene i percorsi di ricerca per i comandi cioè l'elenco delle directory nelle quali la shell cerca il comando da eseguire.

Può essere modificata aggiungendo altri path.

Se la modifichiamo la shell potrebbe non trovare più i comandi:

Esempio:

```
$ echo $PATH
```

```
/bin:/usr/sbin:/usr/bin:/usr/ucb:/etc:/usr/etc:.
```

```
$ PATH=.
```

```
$ echo $PATH
```

```
.
```

```
$ ls
```

```
ls: not found
```



Resta solo la directory corrente

export

Sintassi: *export: export [-n] [name[=value] ...]*
In alternativa *export -p*

Options:

- n remove the export property from each NAME
- p display a list of all exported variables and functions

Permette di ampliare la visibilità alle shell figlie

Se non ci sono argomenti elenca le variabili esportate dalla shell corrente

env : Il comando env permette di avere la lista delle variabili correnti



Esecuzione script con argomenti

Modificare lo script hello world affinché stampi

- * La stringa (composta da tre argomenti) passata in ingresso allo script su tre righe differenti
- * Richieda all'operatore di inserire una durante la sua esecuzione



ESERCIZIO-SOLUZIONE

```
#!/bin/bash  
echo $1  
read STR  
echo $STR  
exit
```

```
#echo della stringa in ingresso  
#richiesta nuova stringa
```



ARRAY

Sintassi: *nomearray=("campo0" "campo1" ..)*

In alternativa:

Nomearray[0]="campo0"

Nomearray[1]="campo1"

nomearray=([0]="campo0" [1]="campo1")

Memorizza oggetti che possono essere richiamati tramite indici numerici

L'indice parte da 0

Esempio di utilizzo: *echo \${nome_array[posizione]}*

@

@ indica intero contenuto

#

restituisce la dimensione dell'array

Dim_array=#nomearray[@]

unset

Rimuove un elemento dell'array

unset nomearray[1]

Elimina l'elemento in posizione 1

NON sposta gli altri

Esecuzione script con array

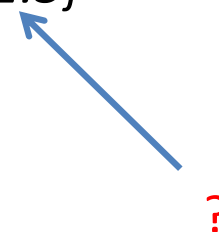
Creare uno script che acquisisca in ingresso 4 parametri inseriti da utente

Carichi i parametri in un array

Stampi a schermo tutti i parametri dell'array

Abbia in coda la riga:

echo "secondo terzo e quarto elemento sono \${nomearray[@]:1:3}"





ESERCIZIO – SLUZION

```
#!/bin/bash
echo "Inserisci il primo parametro:"
read p1
echo "Inserisci il secondo parametro:"
read p2
echo "Inserisci il terzo parametro:"
read p3
echo "Inserisci il quarto parametro:"
read p4

parametri=($p1 $p2 $p3 $p4)      # carica i parametri in un array

echo "I parametri inseriti sono:"
echo ${parametri[@]}           # stampa i parametri dell'array

echo "secondo terzo e quarto elemento sono ${nomearray[@]:1:3}"
```

?



OPERATORI

Operatori aritmetici:

- `+` : somma
- `-` : sottrazione
- `*` : moltiplicazione
- `/` : divisione
- `%` : modulo (resto della divisione)
- `++` : incremento
- `--` : decremento

Operatori logici:

- `&&` : AND logico, restituisce true solo se entrambe le espressioni sono true
- `||` : OR logico, restituisce true se almeno una delle due espressioni è true
- `!` : NOT logico, restituisce true se l'espressione è false

Operatori di assegnazione:

- `=` : trasferisce il valore calcolato
- `+=` , `-=` , `*=` , `/=` , `%=`

Le parentesi tondo per determinare la precedenza degli operatori

BASH supporta soltanto aritmetica intera ($3/2 = 1$)

Per calcolare il risultato va utilizzata la sintassi **`$(( ))`** espansione aritmetica

esempio di somma di due numeri
`echo $(( 2 + 3 ))` # output: 5

Nelle operazioni è possibile riferirsi a variabili dello script

Esempio:

```
var = 5+5  
echo $((5 * var))          => 5*10 =50
```

```
var = 5+5  
echo $((5 * $var))         => 5*5+5=30
```

Possiamo utilizzare l'espansione aritmetica per valutare espressioni booleane,

Gli operatori booleani `&&` e `||` restituiscono

1 per vero

0 per falso:



ARITMETICA NON INTERA

Occorre affidarsi a strumenti esterni es: *bc* (basic calculator)

sudo apt install bc

Esempio:

```
#!/bin/bash
```

```
echo "scale=2; 10/3" | bc      # divide 10 per 3 e stampa il risultato con due cifre decimali
```

```
echo "scale=5; sqrt (10)" | bc #restituisce la radice quadrata di 10 con 5 cifre dopo la virgola
```

Strutture di controllo

Come ogni linguaggio di programmazione procedurale, anche quello della shell dispone di strutture per il controllo dell'esecuzione.

Hanno significati e finalità analoghe a quelli dei tradizionali linguaggi di programmazione

ISTRUZIONE

- o if
- o case
- o for
- o while
- o until

SIGNIFICATO

- selezione/controllo condizionato
- selezione multipla
- iterazione enumerativa
- iterazione condizionata
- iterazione condizionata negata



IF

Comando condizionale

Utilizzato per eseguire un'azione diversa in base al valore di verità di un'espressione condizionale.

Sintassi:

```
if [ espressione]  
then  
    # se l'espressione è vera  
else  
    # se l'espressione è falsa  
fi
```

Possibile annidamento

```
if [ espressione] then  
    if [ espressione] then  
        else  
        fi  
else # se l'espressione è falsa  
    if [ espressione] then  
        else  
        fi  
fi
```

alternativa

```
if [ espressione] then  
  
elif  
  
else  
  
fi
```

Operatori di confronto interi

<i>-eq</i>	: uguale a	<code>==</code>
<i>-ne</i>	: diverso da	<code>!=</code>
<i>-gt</i>	: maggiore di	<code>></code>
<i>-ge</i>	: maggiore o uguale a	<code>>=</code>
<i>-lt</i>	: minore di	<code><</code>
<i>-le</i>	: minore o uguale a	<code><=</code>

Sintassi:

if ["\$a" -ge "\$b"]



spazio

`(( "$a" >= "$b" ))`



Scrivere uno script che richiede all'utente di inserire due numeri.

il programma verifica se i numeri sono
entrambi maggiori di 10 o entrambi minori di 10
stampa un messaggio appropriato a seconda del risultato.

```
#!/bin/bash
```

```
read -p "Inserisci il primo numero: " num1
```

```
read -p "Inserisci il secondo numero: " num2
```

```
if (( ($num1 > 10 && $num2 > 10) || ($num1 < 10 && $num2 < 10) ));
```

```
then
```

```
    echo "Entrambi i numeri sono maggiori o minori di 10"
```

```
else
```

```
    echo "I numeri non hanno lo stesso segno"
```

```
fi
```



case

Comando condizionale

Nel caso in cui gli elif sono molto è più conveniente utilizzare il case

Sintassi:

```
case valore in  
  pattern1)  
    comando1  
    ;;  
  pattern2)  
    comando2  
    ;;  
  pattern3)  
    comando3  
    ;;  
*)  
  comando predefinito di default  
  ;;  
esac
```



Scrivi uno script che chiede di inserire il nome di un mese

Utilizzare il case per visualizzare il numero di giorni in quel mese.

Scrivere 'mese non valido' nel caso non venga inserito il nome di un mese



```
#!/bin/bash
```

```
echo "Inserisci il nome del mese:"  
read mese
```

```
case $mese in  
    gennaio|marchio|maggio|luglio|agosto|ottobre|dicembre)  
        echo "Il mese $mese ha 31 giorni." ;;  
  
    aprile|giugno|settembre|novembre)  
        echo "Il mese $mese ha 30 giorni." ;;  
  
    febbraio)  
        echo "Il mese $mese ha 28 o 29 giorni a seconda dell'anno." ;;  
    *)  
        echo "Il nome inserito non è valido." ;;  
esac
```



for

Ripetizione del comando

Permette di ripetere una determinata azione un certo numero di volte

Sintassi:

```
for variabile in valore1 valore2 ... valoreN  
do  
    comando1  
    ...  
    comandoN  
done
```

Per ogni valore dell'elenco viene impostata la variabile al rispettivo valore ed eseguiti i comandi

Sintassi alternativa che utilizza il ;

```
for (( espress1 ; espress2 ; espress3 ))  
do  
    # Comandi da eseguire  
done
```

espress1: eseguita all'inizio

espress2: condizione di continuazione

espress3: eseguita alla fine



```
for i in 1 2 3 4 5
do
    echo $i
done
```

```
for (( i=1; i<=5; i++ ));
do
    echo $i
done
```

I valori possono essere di qualsiasi tipo

```
for i in 1 2 tre quattro 5 sei
do
    echo "i ha valore $i "
done
```

```
i ha valore 1
i ha valore 2
i ha valore tre
i ha valore quattro
i ha valore 5
i ha valore sei
```



Scrivi uno script che stampi la tavola pitagorica per i numeri da 2 e 3.



```
#!/bin/bash
```

```
for i in {2,3}; do
```

```
# Loop interno per le moltiplicazioni
```

```
for j in {1..10}; do
```

```
    prodotto=$((i*j))    # Calcolo il prodotto e lo stampo
```

```
    echo "$prodotto "
```

```
done
```

```
done
```



while ed until

Ripetizione del comando

While - Permette di ripetere una determinata azione fino a che una condizione è VERA

Sintassi:

```
while condizione
do
    # Comandi da eseguire
done
```

Until - Permette di ripetere una determinata azione fino a che una condizione è FALSA

Sintassi:

```
until condizione
do
    # Comandi da eseguire
done
```



Esempio

Stampa i valori fino a 10

```
i=1  
while [ $i -le 10 ]  
do  
    echo $i  
    i=$((i+1))  
done
```



```
#!/bin/bash
```

```
N=10
```

```
somma=0
```

```
i=1
```

```
while [ $i -le $N ]
```

```
# Esegue il ciclo fintanto che i <= N
```

```
do
```

```
    somma=$((somma+i)) # Aggiungiamo i alla somma
```

```
    i=$((i+1)) #incremento
```

```
done
```

```
echo "La somma dei primi $N numeri è: $somma"
```